

引言

周末想改进一下我的协议SCP，这是一个基于UDP的可靠传输协议。

事情的起因是，我的拥塞控制算法是reno，也就是拿丢包当拥塞，但是我使用netem模拟高丢包网络时，SCP的性能简直拉跨到了极点。

我决定对协议进行一些改进，没想到它莫名其妙性能提升了一大截，而且带宽利用也没变。

跑了好几轮测试，没想到居然很稳定，难道说我乱写代码对了？

真是莫名其妙，我还是记录一下吧。

测试条件、环境、开源项目代码都在我GitHub上，懒得放链接了。

各个条件下的运行结果性能曲线绘图放在文末部分吧。

SB算法

既然写了点东西干脆起个名吧，这个算法就叫**SB算法**好了。

S: **SCP**

B: **Bayesian**

ok，简直完美。

从丢包入手

我在高丢包环境下测试我的SCP协议，但是结果很不满意。

丢包等于拥塞太烦人了。

我当时在思考，协议要怎么做才能识别出，丢包不等于拥塞呢？

一般来说，有这些方法：

1. 看rtt
2. 靠瓶颈探测
3. 看吞吐

但是，比如在wlan场景下，rtt会变，瓶颈甚至也会变，吞吐更是不靠谱。

很多算法使用公式计算拥塞程度，我想了很久，或许可以加上概率。

也就是说，我不打算每一次丢包都老老实实减窗口了，而是靠概率来减。

代价是什么呢？协议会变得激进且有点不公平，无所谓，反正概率公式检查到自己有点激进后，大不了自己退回丢包等于拥塞模型就行了。

那么，让我简单推导一下，我只展示我最终运行比较好的推导近似模型了，其实我参考copa什么的也搞过其他的公式，但是效果不尽人意。

声明

在一栋大厦完成时，应当拆除脚手架。 ----高斯

我近似或者交给参数的地方也比较多，思路也没证明整理，严谨证明太费功夫了，我懒得整理了，当草稿写吧。

推导

条件

链路丢包率

$$p_{\text{link}}$$

满足：

- 1. **非拥塞时**：发送速率不超过容量 (C)，丢包率近似为常数

$$p_{\text{link}} = a$$

- 2. **拥塞时**：这一轮发了 ($B > C$)，观测到丢包率 (b)，且满足

- ($a < b < 1$)
- 拥塞越严重，(b) 越接近 1
- 极端拥塞时，**所有包都死**，($b \rightarrow 1$)

拥塞时的模型

假设这一轮发了 (B) 个包。

引入函数 ($f(B)$)，表示：**这一轮链路最多还能正常处理的包数**

满足：

$$0 \leq f(B) \leq C$$

随着发包量增加，链路越拥塞，($f(B)$) 越小。

情况 1：未超过可处理上限（非拥塞）

如果

$$B \leq f(B)$$

则没有硬丢，只存在背景丢包：

- 背景丢包数：

$$\text{loss} = aB$$

- 丢包率：

$$p_{\text{link}} = \frac{\text{loss}}{B} = a$$

情况 2：超过可处理上限（拥塞）

如果

$$B > f(B)$$

丢包由两部分组成：

① 背景丢包（只作用在可处理的部分）

$$af(B)$$

② 硬丢（超过容量的部分全部丢掉）

$$B - f(B)$$

总丢包数

$$\text{loss}(B) = af(B) + (B - f(B))$$

对应丢包率

$$p_{\text{link}}(B) = \frac{\text{loss}(B)}{B} = \frac{af(B) + (B - f(B))}{B}$$

化简：

$$p_{\text{link}}(B) = 1 - \frac{(1-a)f(B)}{B}, \quad B > f(B)$$

最终有

$$p_{\text{link}}(B) = \begin{cases} a, & B \leq f(B) \\ 1 - \frac{(1-a)f(B)}{B}, & B > f(B) \end{cases}$$

该公式满足：

- 非拥塞：plink= a)
- 拥塞：(a < plink< 1)
- 极端拥塞：f(B) -> 0, plink -> 1

f(B)到底是多少呢？

根据时间局部性，上一次发生的事情大概率会延续到下一次。

比如你昨天去上班，今天大概率也会去上班，昨天去上学，今天大概率也是去上学。转变是少数，大部分时候，我们都会重复过去一段时间的状态。

B真的会影响f(B)吗？在单流场景会有，但是不多，因为反正快拥塞了，都很小。在多流场景下，影响就更加小了。

假设上一次可处理数据包与这一次可处理的相似，那么f(B)可近似看成常数。

严肃说法就是：在短时间窗口内，瓶颈链路的服务速率由物理带宽与调度策略决定，而这些因素的变化时间尺度通常远大于 RTT 级别的观测窗口，因此可以近似认为服务函数 f(B) 在局部窗口内为常数。

观测量定义

定义加权平均参数，计算丢包率，SCP协议需要在一个rtt内统计丢包率。

$$\hat{p} = \frac{L}{N}$$

然后定义rtt的变化，统计最小rtt并加权为rtt_base，这其实是参考了copa的模型：

$$r = \frac{\text{srtt}}{\text{rtt}_{base}}$$

计算变化率：

$$d = r - 1 = \frac{\text{srtt} - \text{rtt}_{base}}{\text{rtt}_{base}}$$

贝叶斯

现在，我要通过两个信号，丢包率和rtt，得到拥塞相关的概率。

全概率贝叶斯公式：

$$P(A | B) = \frac{P(B | A) P(A)}{\sum_i P(B | A_i) P(A_i)}$$

我们是二元假设，也就是：

其中1代表是，0就是不是的概率。

$$P(A_1 | B) = \frac{P(B | A_1) P(A_1)}{P(B | A_1) P(A_1) + P(B | A_0) P(A_0)}$$

写出相反事件的后验

$$P(A_0 | B) = \frac{P(B | A_0) P(A_0)}{P(B | A_1) P(A_1) + P(B | A_0) P(A_0)}$$

相除后得到后验比：

$$\frac{P(A_1 | B)}{P(A_0 | B)} = \frac{P(B | A_1) P(A_1)}{P(B | A_0) P(A_0)}$$

也就是：

$$\frac{P(A_1 | B)}{P(A_0 | B)} = \frac{P(A_1)}{P(A_0)} \cdot \frac{P(B | A_1)}{P(B | A_0)}$$

取对数，变成线性可加形式

$$z = \log \frac{P(A_1 | B)}{P(A_0 | B)} = \log \frac{P(A_1)}{P(A_0)} + \log \frac{P(B | A_1)}{P(B | A_0)}$$

因为：

$$P(A_0 | B) = 1 - P(A_1 | B)$$

所以：

$$\frac{P(A_1 | B)}{1 - P(A_1 | B)} = e^z$$

解得：

$$P(A_1 | B) = \frac{1}{1 + e^{-z}}$$

丢包

丢包事件是服从二项分布的，

我们知道二项分布的概率质量函数是：

$$P(K | N, p) = \binom{N}{K} p^K (1 - p)^{N-K}$$

根据我们上面的模型，有

非拥塞

$$p = a$$

$$P(K | A_0) = \binom{N}{K} a^K (1 - a)^{N-K}$$

拥塞

那么，假设概率为 $p_C(B)$ ：

$$p = p_C(B)$$

$$P(K | A_1) = \binom{N}{K} p_C(B)^K (1 - p_C(B))^{N-K}$$

如果稳定拥塞，那么对应的sent/loss比例也可以看成是线性增长：

其实这里无关紧要，还是写一下吧，后面我可能想更加精细化：

$$p = (a1 * t + a)$$

$$P(K | A_1) = \binom{N}{K} p^K (1 - p_C(B))^{N-K}$$

代入

把贝叶斯中的观测项换成丢包事件：

$$\log \frac{P(B | A_1)}{P(B | A_0)} \Rightarrow \log \frac{P(K | A_1)}{P(K | A_0)}$$

算 log 似然比

$$\frac{P(K | A_1)}{P(K | A_0)} = \frac{\binom{N}{K} p_C(B)^K (1 - p_C(B))^{N-K}}{\binom{N}{K} a^K (1 - a)^{N-K}}$$

组合数相同，直接消掉：

$$\frac{P(K | A_1)}{P(K | A_0)} = \left(\frac{p_C(B)}{a} \right)^K \left(\frac{1 - p_C(B)}{1 - a} \right)^{N-K}$$

取对数：

$$\log \frac{P(K | A_1)}{P(K | A_0)} = K \log \frac{p_C(B)}{a} + (N - K) \log \frac{1 - p_C(B)}{1 - a}$$

换成丢包率公式

$$K = N\hat{p}, \quad N - K = N(1 - \hat{p})$$

代入：

$$\log \frac{P(K | A_1)}{P(K | A_0)} = N \left[\hat{p} \log \frac{p_C(B)}{a} + (1 - \hat{p}) \log \frac{1 - p_C(B)}{1 - a} \right]$$

原来的：

$$z = \log \frac{P(A_1)}{P(A_0)} + \log \frac{P(B | A_1)}{P(B | A_0)}$$

现在只用丢包观测 (K)，就是：

$$z_{\text{loss}} = \log \frac{P(A_1)}{P(A_0)} + N \left[\hat{p} \log \frac{p_C(B)}{a} + (1 - \hat{p}) \log \frac{1 - p_C(B)}{1 - a} \right]$$

做近似

我们知道 (A) 表示链路是否处于拥塞状态，这部分考虑现实混沌影响，看成常数吧，毕竟无法建模也是一种建模，而 (p_C(B)) 在短时间窗口内可以近似为常数。

因此，丢包的 log 似然比部分：

$$N \left[\hat{p} \log \frac{p_C(B)}{a} + (1 - \hat{p}) \log \frac{1 - p_C(B)}{1 - a} \right]$$

可以视为关于 p 的一次函数。

把所有常数项放到一边，工作点附近的一阶分数近似：

$$z = \gamma_0(B) + \alpha(B) \hat{p}$$

独立性假设

如果假设 **丢包 (K)** 和 **RTT (d)** 在给定的链路状态 A 的条件下是独立的（真的成立吗？为了方便推导，假设吧），那么有：

$$P(K, d | A_i) = P(K | A_i) P(d | A_i)$$

使用的后验比是：

$$\frac{P(A_1 | K, d)}{P(A_0 | K, d)} = \frac{P(A_1)}{P(A_0)} \cdot \frac{P(K, d | A_1)}{P(K, d | A_0)}$$

利用独立性展开：

$$\frac{P(K, d | A_1)}{P(K, d | A_0)} = \frac{P(K | A_1)}{P(K | A_0)} \cdot \frac{P(d | A_1)}{P(d | A_0)}$$

取对数后变成加法：

$$z = \log \frac{P(A_1 | K, d)}{P(A_0 | K, d)}$$

代入上式得到：

$$z = \log \frac{P(A_1)}{P(A_0)} + \log \frac{P(K | A_1)}{P(K | A_0)} + \log \frac{P(d | A_1)}{P(d | A_0)}$$

接下来推导rtt:

RTT 的贝叶斯推导

假设 RTT 膨胀量 (d) 在给定链路状态下服从指数分布:

- 非拥塞:

$$d | A_0 \sim \text{Exp}(\lambda_0)$$

- 拥塞:

$$d | A_1 \sim \text{Exp}(\lambda_1)$$

指数分布密度为:

$$P(d | A_i) = \lambda_i e^{-\lambda_i d}, \quad d \geq 0$$

写出 RTT 的似然比

$$\frac{P(d | A_1)}{P(d | A_0)} = \frac{\lambda_1 e^{-\lambda_1 d}}{\lambda_0 e^{-\lambda_0 d}} = \frac{\lambda_1}{\lambda_0} e^{-(\lambda_1 - \lambda_0)d}$$

取对数:

$$\log \frac{P(d | A_1)}{P(d | A_0)} = \log \frac{\lambda_1}{\lambda_0} - (\lambda_1 - \lambda_0)d$$

我们不假设 RTT 遵循特定的分布, 我们假设它的对数密度在拥塞状态下是局部线性的。

写成线性形式

定义:

$$\beta_0 := \log \frac{\lambda_1}{\lambda_0}, \quad \beta := -(\lambda_1 - \lambda_0)$$

于是可以写成:

$$\log \frac{P(d | A_1)}{P(d | A_0)} = \beta_0 + \beta d$$

同时有:

$$\beta d = \beta \cdot \frac{\text{srtt} - \text{rtt}_{base}}{\text{rtt}_{base}}$$

最终

总的 log-odds:

$$z = \log \frac{P(A_1)}{P(A_0)} + \log \frac{P(K | A_1)}{P(K | A_0)} + \log \frac{P(d | A_1)}{P(d | A_0)}$$

将丢包部分与 RTT 部分代入:

$$z = \underbrace{\left(\log \frac{P(A_1)}{P(A_0)} + \gamma_0(B) + \beta_0 \right)}_{\gamma} + \alpha(B) \hat{p} + \beta d$$

最终得到线性模型：

$$z = \gamma + \alpha(B) \hat{p} + \beta d$$

拥塞概率

$$P(A_1 | \hat{p}, d, B) = \frac{1}{1 + e^{-(\gamma + \alpha(B) \hat{p} + \beta d)}}$$

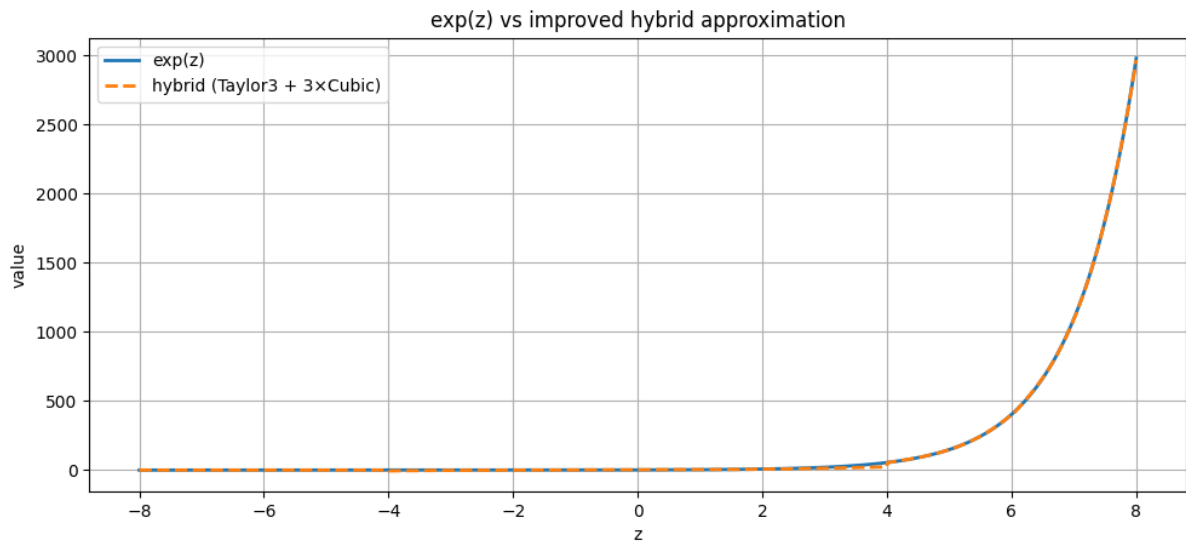
代码实现

这里面太多公式了，为了不引入数学库和浮点数，只能放缩了。

经过尝试，我还是选择了二阶展开exp，原本我是想用三阶泰勒展开加cubic逼近的，因为这样精度很高。

但问题就是，计算太复杂了，性能很差，其实二阶也够用了。

下面是三阶泰勒展开加cubic逼近，这已经非常完美了，可惜了，这个分段逼近是真复杂，还是用最简单的二阶吧。



最终的代码实现：

先用浮点数，确定范围后再思考放缩和逼近的代码实现，其实最早我想用多段查表，但感觉没必要。

我的实现合理吗？

不合理的，其实我也知道，丢包与srtt其实相关性非常大，独立性很难成立，而且也缺乏对物理世界的观测，这个模型还需要进一步打磨。

但是，为了尽快跑起来，代码进行了妥协，用magic number试试效果吧。

```
//if you want to know why, please see docs.
#define Q16_ONE    (1 << 16)
#define Q16_INT(x) ((int32_t)((x) << 16))
#define Q16(x) ((int32_t)((x) << 16))
```



```

#define CONFIG_P (1 << 15)

#define NEG8      Q16(-8)
#define POS8      Q16(8)

static const uint16_t ALPHA = 8;
static const uint16_t BETA  = 4;
static const int32_t  GAMMA = Q16_INT(-3);

//I think this is ok for [-8, 8].
static inline int32_t exp_q16(int32_t z)
{
    if (z < NEG8) z = NEG8;
    if (z > POS8) z = POS8;

    int32_t z2 = (int32_t)(((int64_t)z * z) >> 17);
    return Q16_ONE + z + z2;
}

static inline uint16_t logistic(int32_t z)
{
    if (z > Q16(20)) z = Q16(20);
    if (z < Q16(-20)) z = Q16(-20);

    int32_t e = exp_q16(-z);
    int32_t d = 65536 + e;
    int32_t q = (int32_t)(((int64_t)65536 << 16) / d);

    if (q > 65535) q = 65535;
    if (q < 0) q = 0;
    return (uint16_t)q;
}

static inline void scp_md_prob(struct scp_stream *ss)
{
    if (ss->sent_cnt == 0)
        return;

    ss->p = (uint16_t)(((uint64_t)ss->loss_cnt << 16) / ss->sent_cnt);

    if (ss->rtt_base > 0 && ss->srtt > 0) {
        int32_t ratio = (int32_t)(((int64_t)ss->srtt << 16) / ss->rtt_base);
        ss->d = ratio - Q16_ONE;
        if (ss->d < 0) ss->d = 0;
    } else {
        ss->d = 0;
    }

    ss->z = GAMMA
        + (int32_t)((int64_t)ALPHA * ss->p)
        + (int32_t)((int64_t)BETA * ss->d);

    ss->cong_q = logistic(ss->z);
}

```

```
static inline int scp_loss_like_congestion(struct scp_stream *ss)
{
    scp_md_prob(ss);
    return ss->cong_q > CONFIG_P;
}
```

性能如何呢？

提升非常大，甚至让我很意外，但是好像也没有那么大。

双向互传100MB大文件测试。

接下来对比两个版本，丢包等于拥塞版本和SB算法版本。

netem条件：

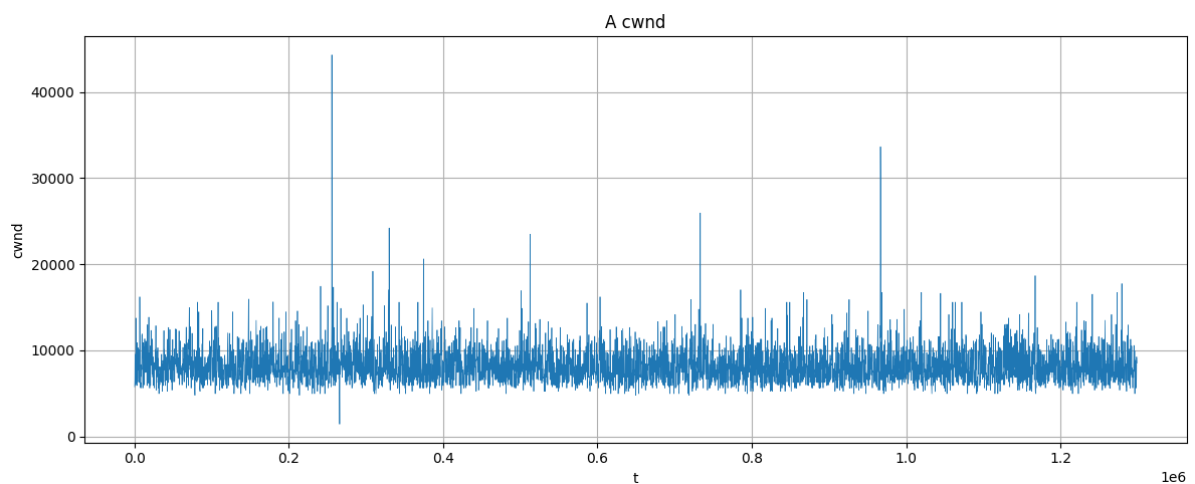
我只改动丢包率，从0%一路到10%。

```
skaiuijing:~/Documents/SCP/test/nodeA/build$ sudo tc qdisc show
qdisc htb 1: dev lo root refcnt 2 r2q 10 default 0x10 direct_packets_stat 0
direct_qlen 1000
qdisc netem 10: dev lo parent 1:10 limit 50 delay 50ms 10ms loss 10%
qdisc fq_code1 0: dev ens33 root refcnt 2 limit 10240p flows 1024 quantum 1514
target 5ms interval 100ms memory_limit 32Mb ecn drop_batch 64
```

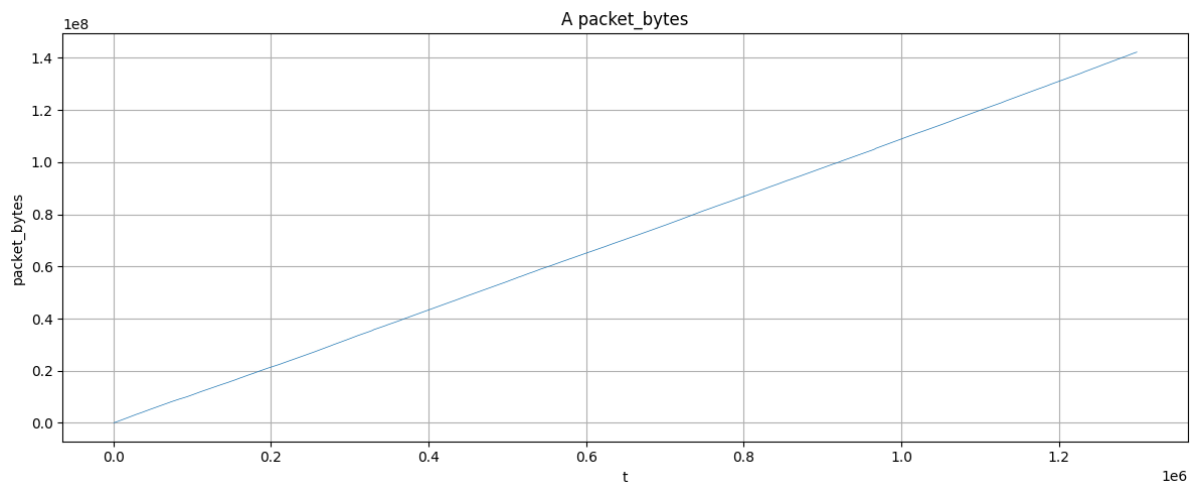
不同的丢包率下测试对比

丢包率0%

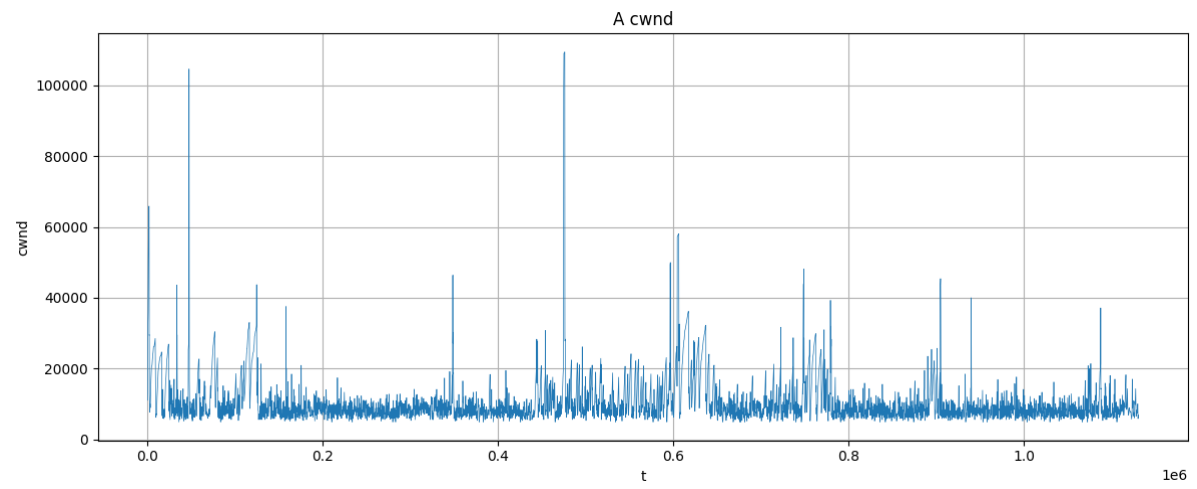
丢包==拥塞版本



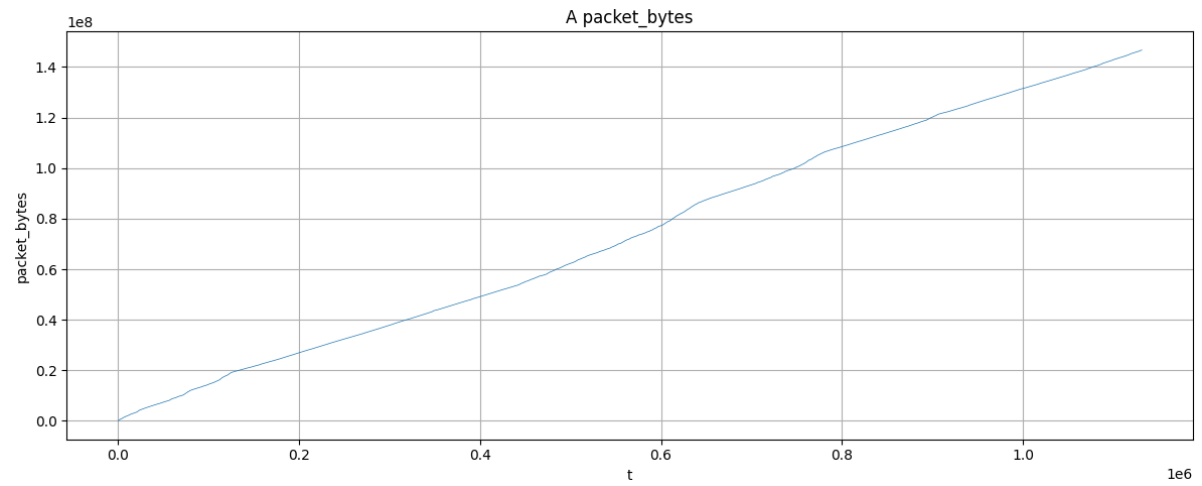
字节传输曲线：



SB算法版本

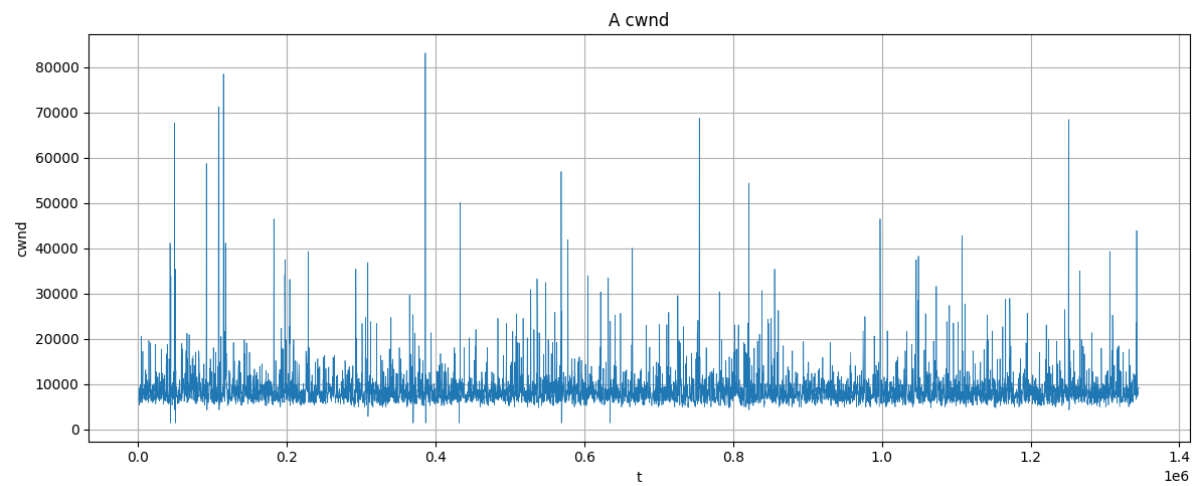


传输:

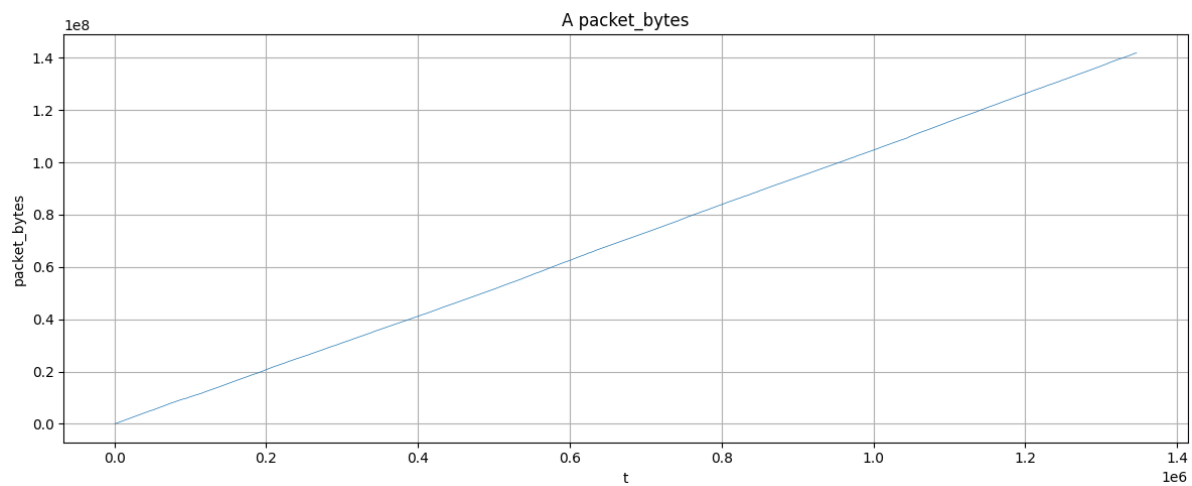


丢包率2%

丢包==拥塞版本：

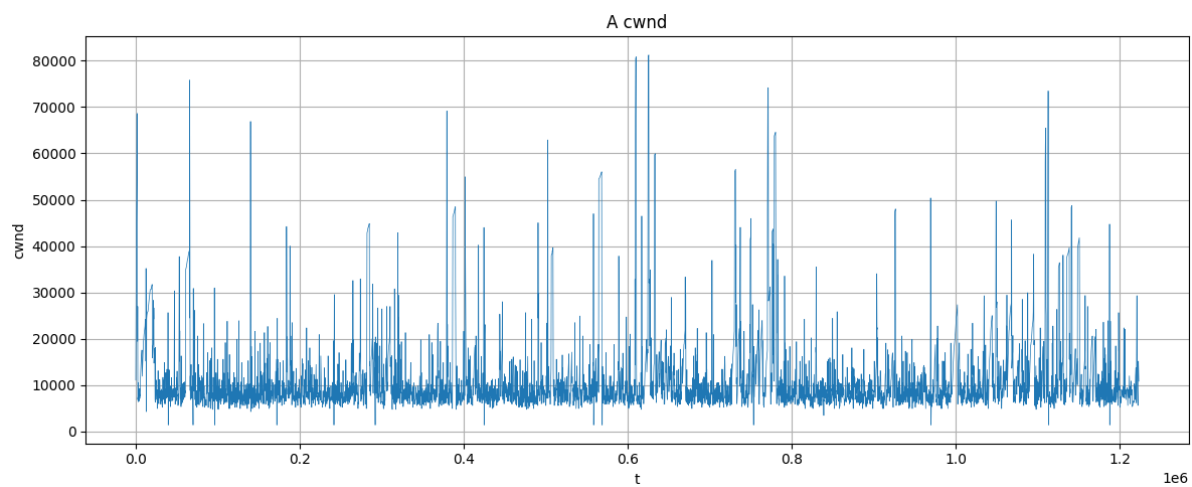


字节传输曲线：

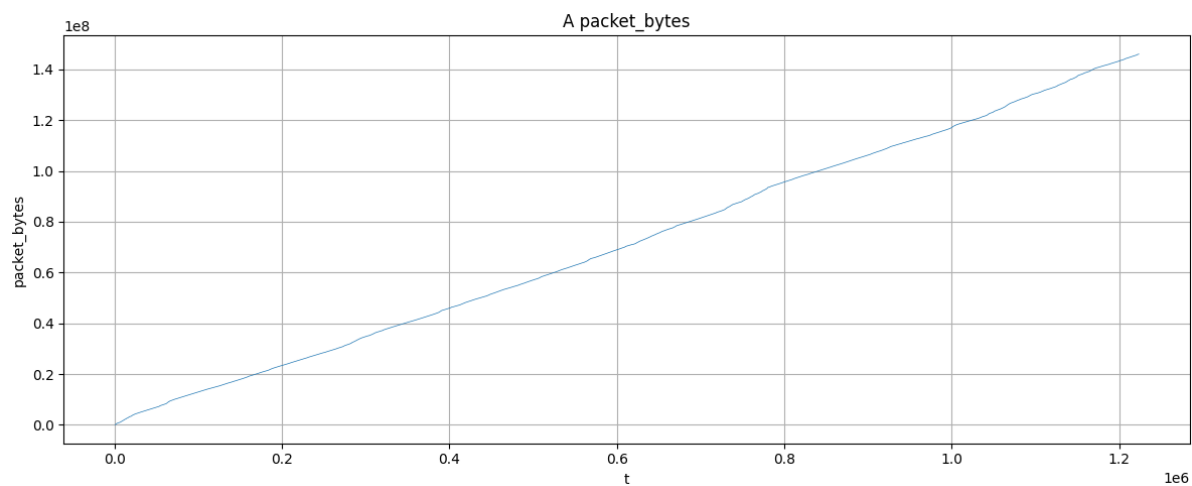


SB算法

快了很多，其实你会发现，下面的曲线有锯齿状的感觉了，如果放大观察，会更加明显。

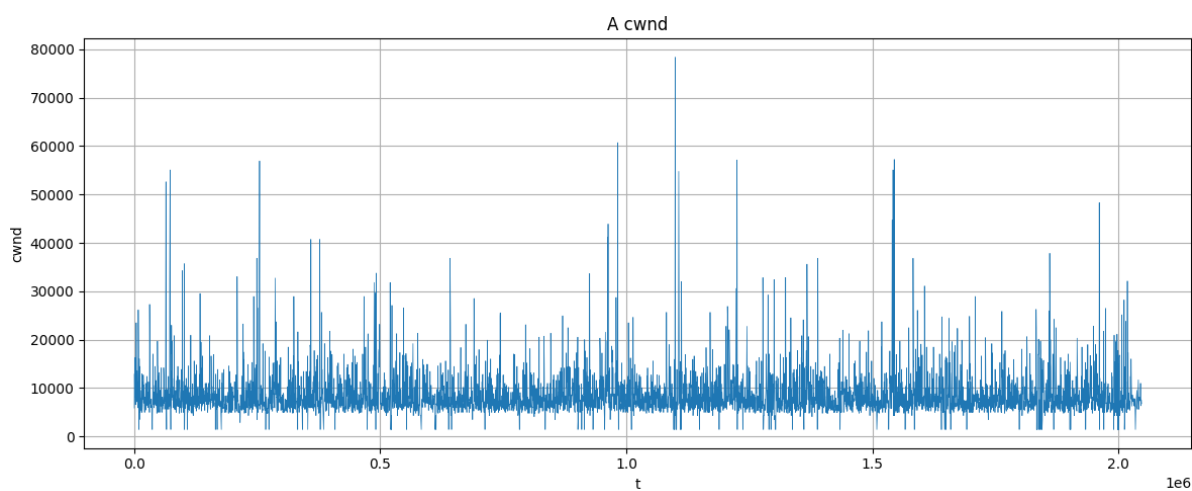


有点激进：

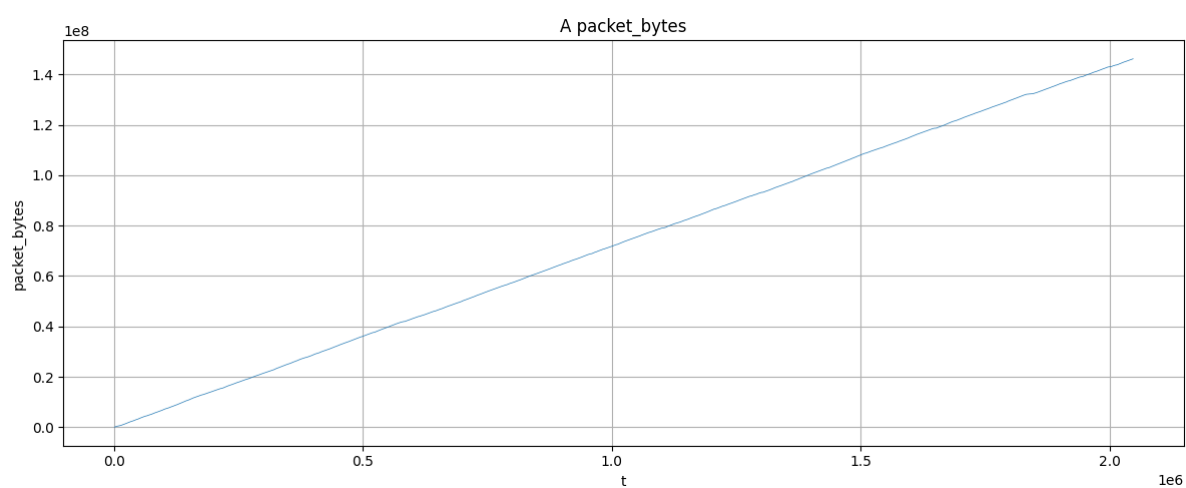


丢包率10%

丢包==拥塞版本



字节传输曲线:

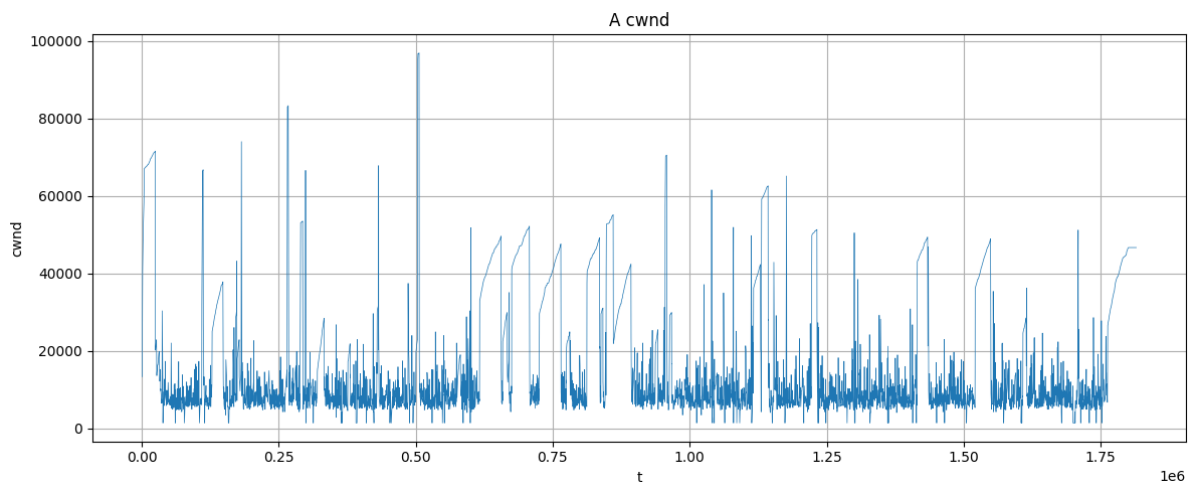


SB算法版本

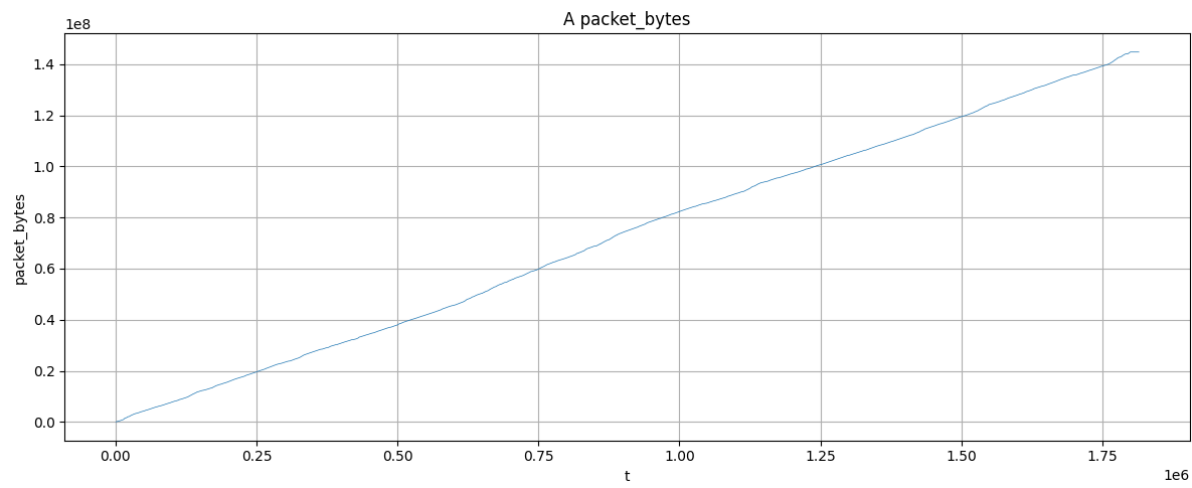
拥塞窗口曲线：

看看，对比之前的模型，太美观了，好看太多了。

美感不仅仅是反映系统运行状态的指标，这样的另一个好处是，如果对曲线求积分，可以得到总吞吐。



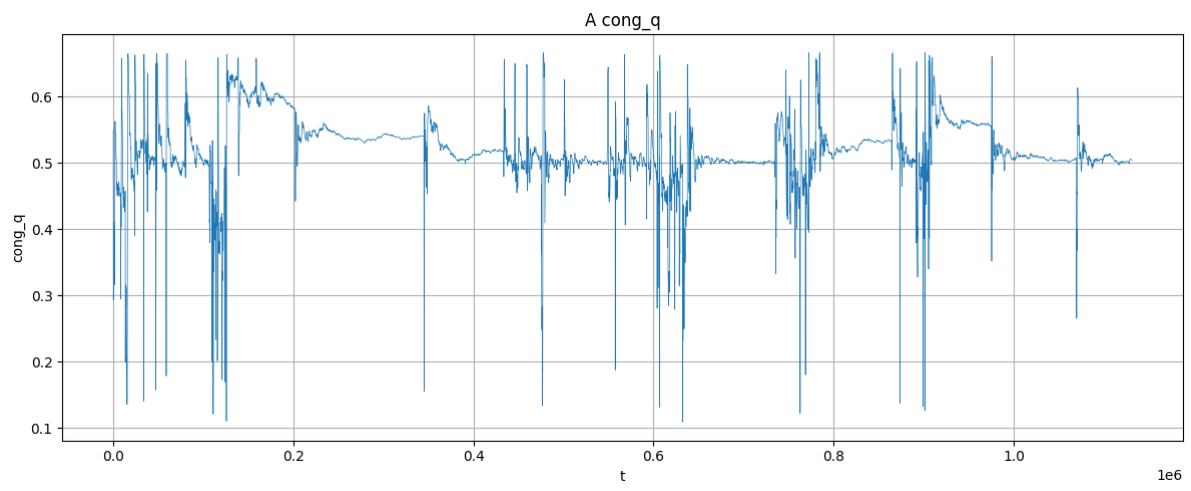
传输：



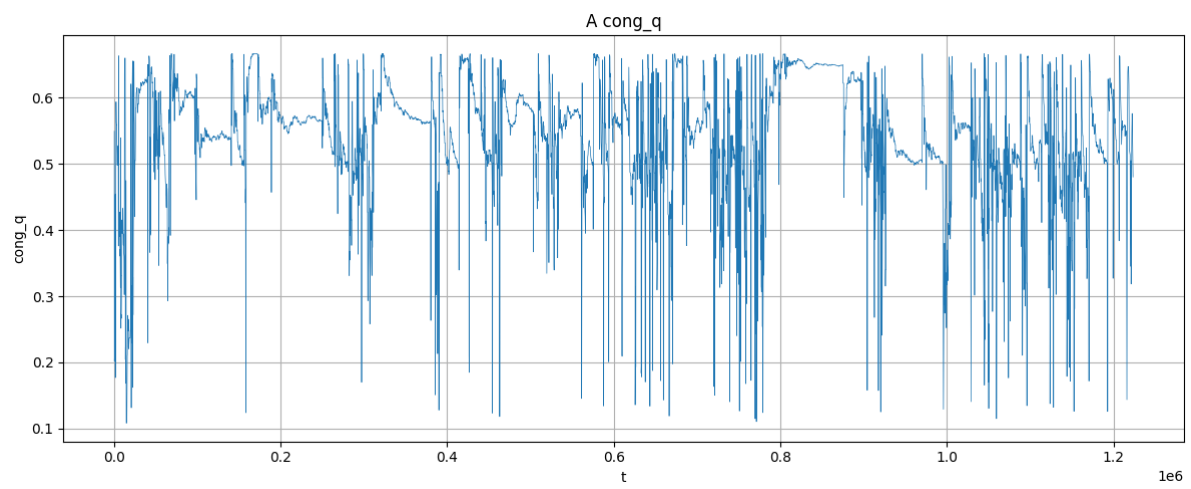
概率曲线

为什么会跑成这样呢？

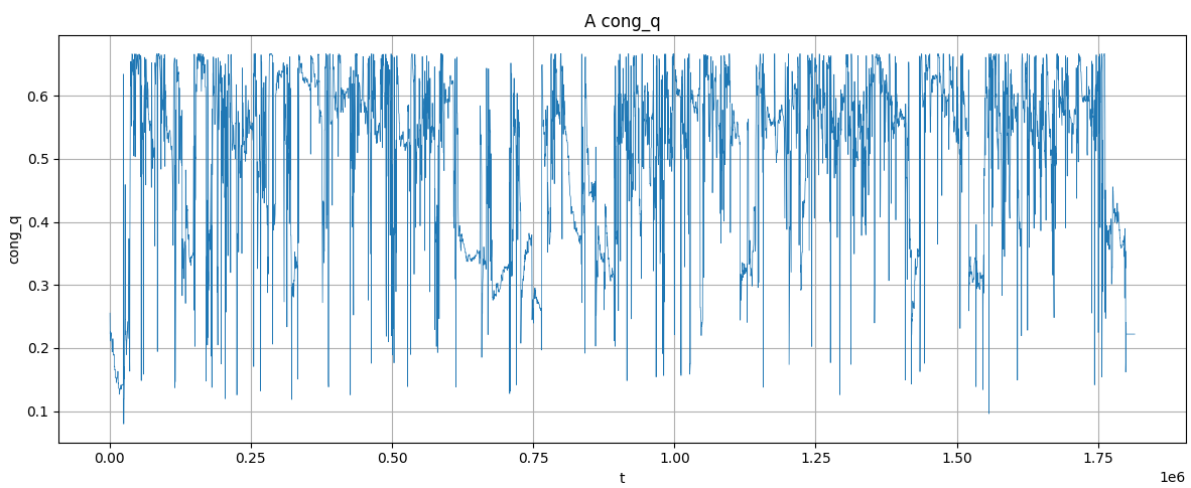
0%：



2%丢包:



10%:



最后

模型还需要改进，工程实现还很粗糙。

不过，感觉勉强能用？